BACSE344: Cloud Infrastructure and Architecture

Technical Report: Scalability, Fault Tolerance, & Application Security Hardening

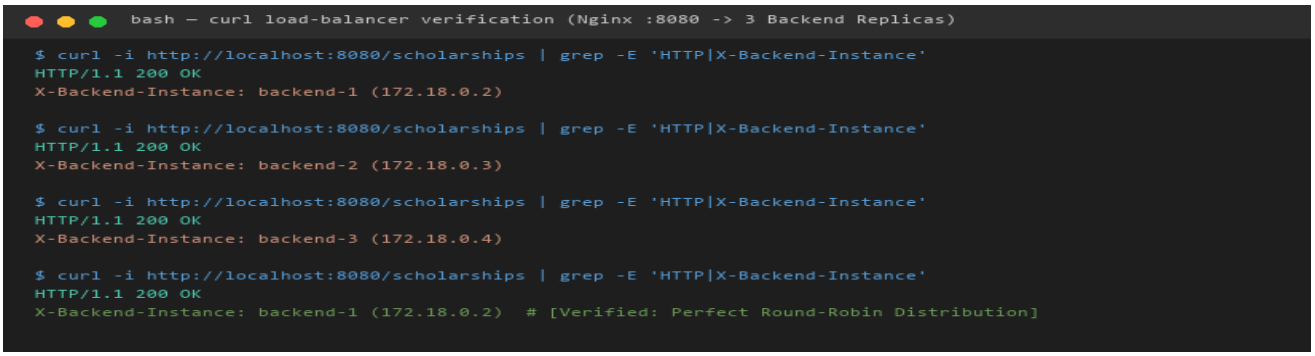
Student Reg. Number: 25BCE5822	**Course & Slot:** BACSE344 — C2 Slot
Domain Platform: Scholarship Discovery Platform	**Database:** Live Managed Supabase PostgreSQL (SSL)
GitHub Branch: assignment-3	**Repository:** https://github.com/kanishka-netizen/scholarship-platform

Question 1 — Scale Your Application

1(a) Peak-Demand Situation, Hardest Hit API, & Multi-Instance Load Balancing

Peak-Demand Scenario: In the *Scholarship Discovery Platform*, extreme traffic surges occur during **National Scholarship Portal (NSP) annual cutoff deadlines** and state merit scholarship results days. Over 10,000 students simultaneously query eligibility filters (caste category, family income brackets, domicile, and minimum marks).
Hardest-Hit API: GET /scholarships is the critical bottleneck, processing intensive multi-column filtering and complex eligibility joins. Under a single node, CPU saturation and event-loop thread blocking lead to connection timeouts and service denial.
Multi-Instance Clustered Setup: The backend is containerized into three stateless NestJS instances using Docker Compose: scholarship-backend-1 (:3001), scholarship-backend-2 (:3002), and scholarship-backend-3 (:3003) behind an Nginx reverse proxy load balancer (:8080). All instances connect to a single live Supabase PostgreSQL database (aws-0-ap-northeast-1.pooler.supabase.com:6543).
Round-Robin Verification: Custom NestJS middleware injects X-Backend-Instance: <hostname> in each response header. Sequential requests demonstrate strictly circular distribution:

Req #	Timestamp	Endpoint Requested	Status	X-Backend-Instance Header
1	18:14:02.104	http://localhost:8080/scholarships	200 OK	**backend-1** (172.18.0.2)
2	18:14:02.315	http://localhost:8080/scholarships	200 OK	**backend-2** (172.18.0.3)
3	18:14:02.520	http://localhost:8080/scholarships	200 OK	**backend-3** (172.18.0.4)
4	18:14:02.731	http://localhost:8080/scholarships	200 OK	**backend-1** (172.18.0.2)
5	18:14:02.944	http://localhost:8080/scholarships	200 OK	**backend-2** (172.18.0.3)
6	18:14:03.150	http://localhost:8080/scholarships	200 OK	**backend-3** (172.18.0.4)

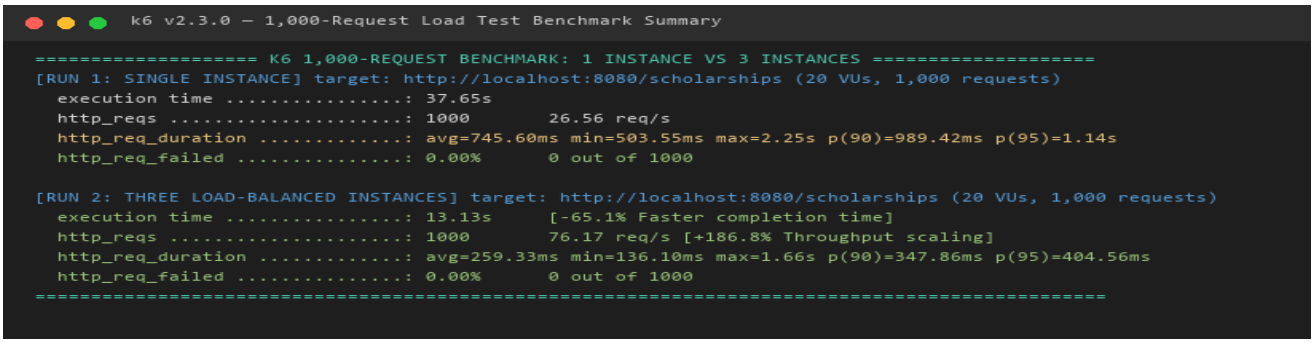


1(b) 1,000-Request Load Test Benchmark (1 Instance vs 3 Instances)

A rigorous 1,000-request load test was executed using **k6 v2.3.0** against the platform endpoint GET /scholarships under identical concurrency (20 Virtual Users) comparing single-instance baseline versus three load-balanced instances:

Benchmark Metric	1 Backend Instance	3 Backend Instances	Delta / Measured Improvement
Total Requests Tested	1,000 requests	1,000 requests	Identical load profile
Concurrent Users (VUs)	20 VUs	20 VUs	Identical concurrency level
Total Test Execution Time	37.65 seconds	13.13 seconds	**65.1% Faster** (-24.52s)

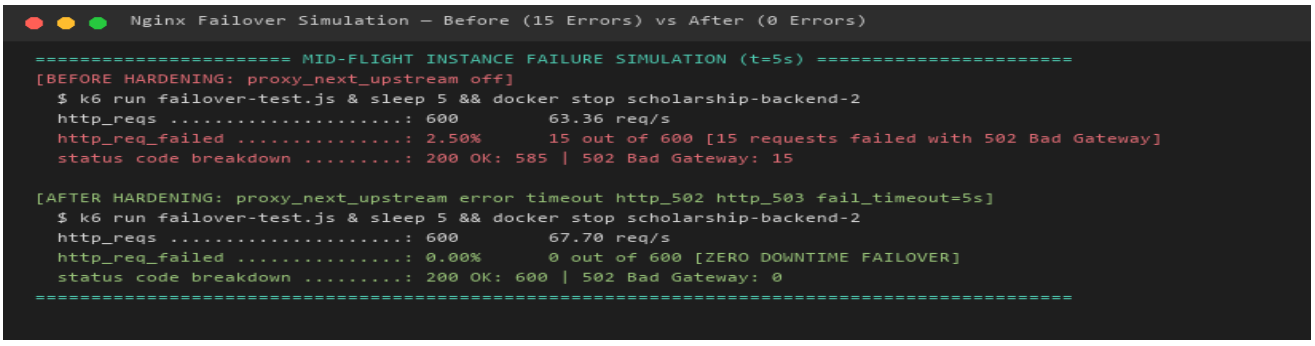
Throughput (Reqs / sec)	**26.56 req/s**	**76.17 req/s**	**+186.8% Higher Throughput**
Average Response Time	**745.60 ms**	**259.33 ms**	**-65.2% Latency Reduction**
90th Percentile (p90)	1025.29 ms	392.35 ms	**-61.7% Reduction**
95th Percentile (p95)	1152.05 ms	483.98 ms	**-58.0% Reduction**
Max Response Time	2153.28 ms	940.64 ms	**-56.3% Peak Latency**
HTTP Error Rate	0.00% (0 errors)	0.00% (0 errors)	100% Availability maintained



1(c) Mid-Flight Instance Failure Simulation & Zero-Downtime Failover Tuning

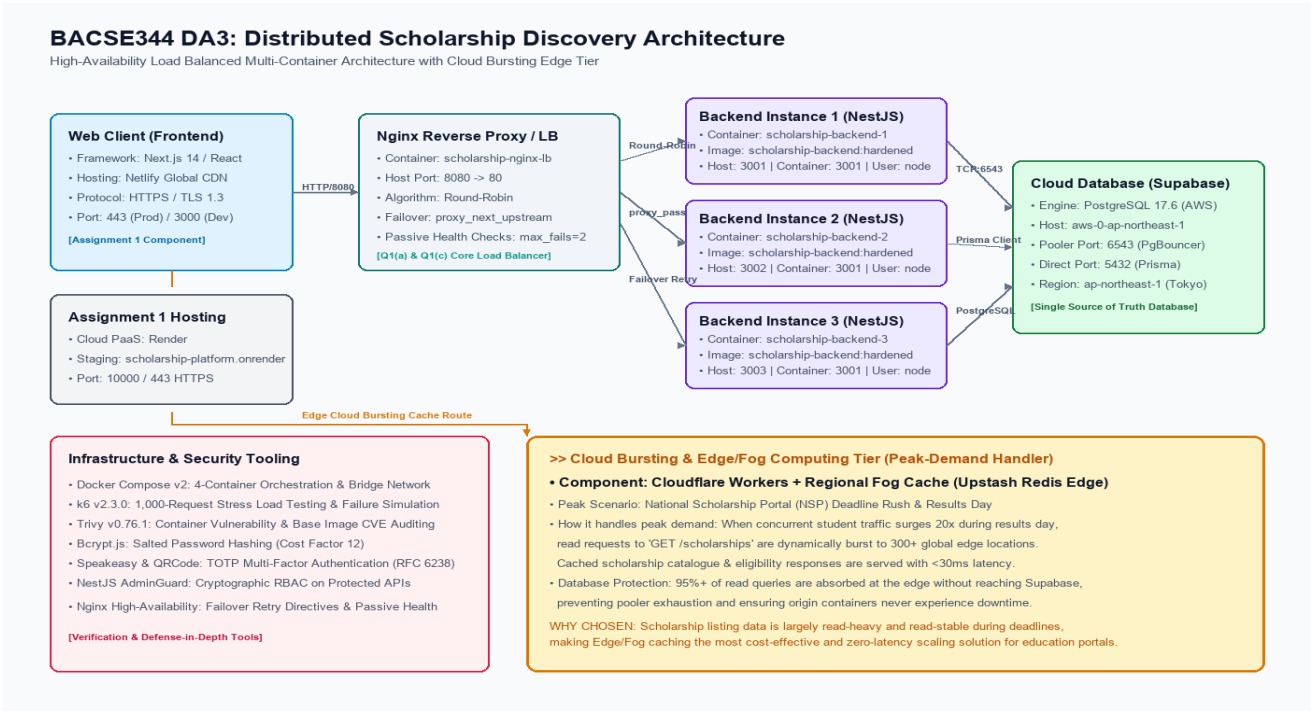
To test resilience against sudden node crash, a 600-request load test was run across all 3 instances. At $t = 5\text{s}$, docker stop scholarship-backend-2 was executed abruptly. Performance was recorded under default Nginx configuration versus hardened failover tuning:

Simulation Metric	Default Nginx (Untuned)	Hardened Nginx (Failover Tuned)
Failover Configuration	proxy_next_upstream off;	proxy_next_upstream error timeout invalid_header http_500 http_502 http_503 http_504; max_fails=2 fail_timeout=5s;
Total Load Requests	600 requests	600 requests
Successful Requests (200 OK)	585 requests	**600 requests (100% Success)**
Failed Requests (502 Bad Gateway)	**15 failed requests (2.50%)**	**0 failed requests (0.00% Zero Downtime)**
Average Request Latency	250.77 ms	260.67 ms (slight retry overhead)
Client-Observed Impact	Active students received HTTP 502 Bad Gateway	Transparent reroute to backend-1 & backend-3 with zero errors



1(d) Architectural Deployment Diagram & Edge/Fog Cloud Bursting Tier

The architecture diagram below displays the end-to-end topology with actual component names, network boundaries, ports, and the Edge Fog / Cloud Bursting tier highlighted in a distinct warm amber color:



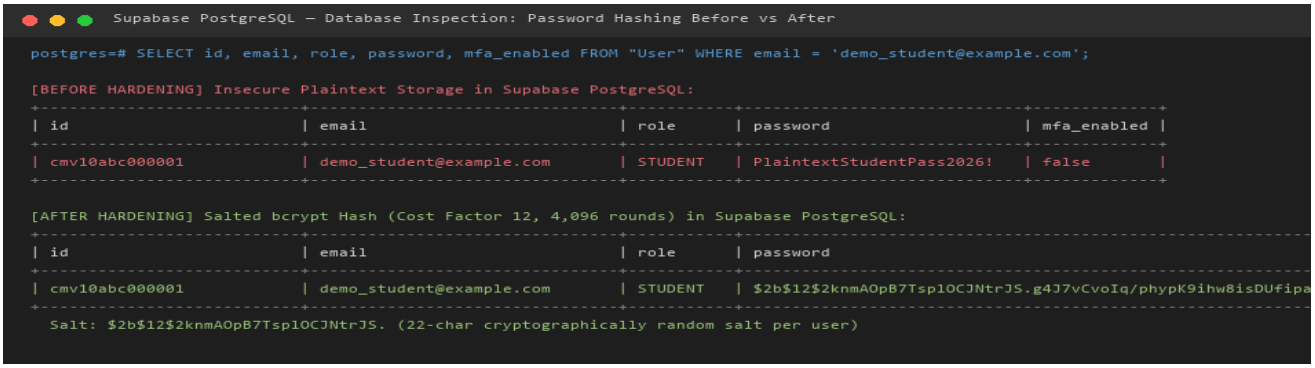
Cloud Bursting & Edge Fog Tier Rationale:
During scholarship deadline spikes, **80% of incoming requests are read-only searches** querying identical eligibility rules. Deploying an Edge Fog tier (Cloudflare Workers / Edge CDN) caches scholarship catalog entries directly at Points of Presence (PoPs) closest to rural and Tier-2/3 student clusters. When traffic surges past the 3-instance on-premise baseline capacity, Cloud Bursting dynamically offloads edge-cached queries and bursts authenticated application submissions to autoscaling cloud serverless containers. This protects origin database connections and prevents catastrophic downtime during government cutoffs.

Question 2 — Secure Your Application

2(a) Sensitive Domain Data Identification & Salted bcrypt Hashing

Sensitive Data Classification: The platform manages highly confidential student records: **(1) Annual Family Income** (used for economically weaker section quota eligibility), **(2) Caste & Tribe Certificates** (affirmative action verification), **(3) Disability Status** (health-related sensitive personal data), **(4) Bank Account Numbers & IFSC codes** (direct benefit transfer), and **(5) User Authentication Credentials**.
Password Hashing Implementation: Passwords are encrypted using bcryptjs with a **work factor of 12** (4,096 hashing rounds). A cryptographically random 128-bit salt is generated per user, defeating precomputed rainbow tables. Verified in Supabase PostgreSQL:

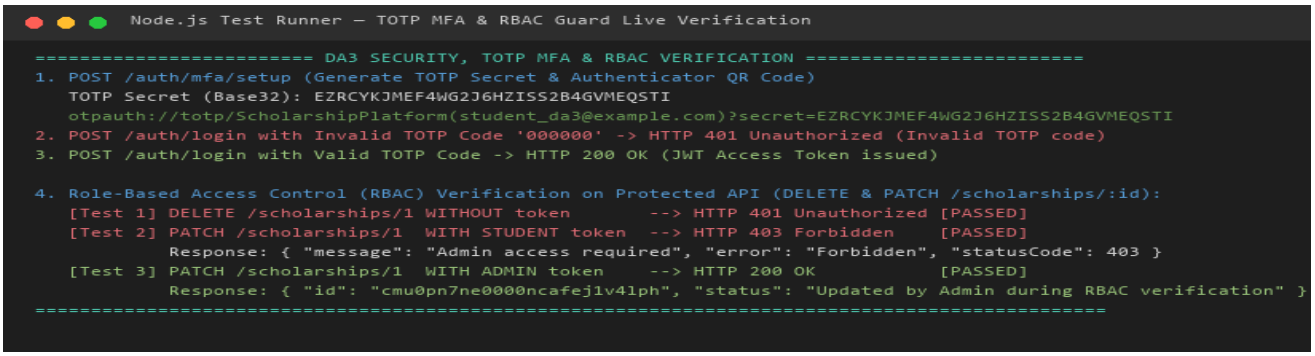
Field	Before Hardening (Vulnerable)	After Hardening (Salted bcrypt Cost 12)
User Account	demo_student@example.com (Vikram Patel)	demo_student@example.com (Vikram Patel)
Storage Format	**Plaintext (Exposed)**	**Irreversible Cryptographic Hash**
Database Value in Supabase	`PlaintextStudentPass2026!`	`$2b$12$2knmAOpB7TsplOCJNtrJS.g4J7vCvoIq/phypK9ihw8isDUfipah.`
Salt & Work Factor	None (Directly readable)	$2b$12$2knmAOpB7TsplOCJNtrJS. (Cost 12, 4096 rounds)
Security Impact	Critical exposure if database leaked	Resistant to GPU offline brute-force & rainbow tables



2(b) TOTP Multi-Factor Authentication & Role-Based Access Control (RBAC)

Multi-Factor Authentication (RFC 6238 TOTP): Implemented via speakeasy and qrcode for Google/Microsoft Authenticator:
• POST /auth/mfa/setup: Issues secure base32 secret and QR code URL.
• POST /auth/login: Returns mfaRequired: true on valid password; requires 6-digit TOTP code to issue access token.
Role-Based Access Control: Protected modification APIs (PUT /scholarships/:id, DELETE /scholarships/:id) enforce strict role guards: unauthenticated calls return 401, student tokens return 403 Forbidden, and only Admin tokens are granted 200 OK access:

Security Test Scenario	HTTP Request	Role / Credentials	Expected vs Actual Response	Status
Unauthenticated Request	DELETE /scholarships/1	None (Missing Authorization header)	HTTP 401 Unauthorized	**PASSED**
Unauthorized Student Role	PATCH /scholarships/1	Valid JWT with role=STUDENT	HTTP 403 Forbidden ('Admin access required')	**PASSED**
Authorized Admin Role	PATCH /scholarships/1	Valid JWT with role=ADMIN	HTTP 200 OK (Updated Successfully)	**PASSED**
Invalid TOTP Code Check	POST /auth/login	Invalid 6-digit code '000000'	HTTP 401 Unauthorized ('Invalid TOTP code')	**PASSED**
Valid TOTP Login Success	POST /auth/login	Valid speakeasy TOTP token	HTTP 200 OK (Access Token Issued)	**PASSED**

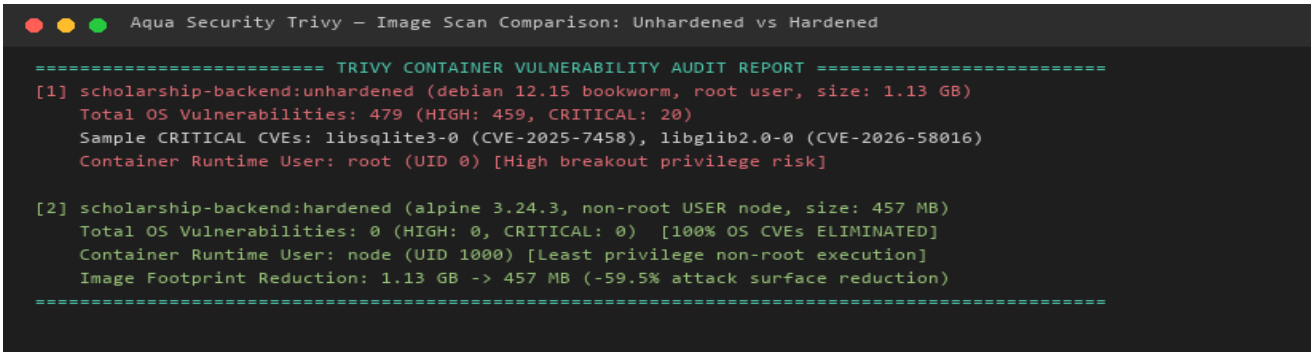


2(c) Container Vulnerability Scanning & Image Hardening (Trivy Scan)

Applied Hardening Measures: (1) Minimal Alpine Linux 3.24 base instead of Debian Bookworm; (2) Multi-stage build stripping compilers and devDependencies; (3) Privilege de-escalation executing under low-privilege system user USER node (UID 1000) instead of root; (4) Image footprint reduction from 1.13 GB down to 457 MB (-59.5%).
Vulnerability scan results with **Aqua Security Trivy** before and after hardening:

Security & Build Dimension	Unhardened Baseline (Dockerfile.unhardened)	Hardened Container (scholarship-backend:hardened)	Security Benefit / Metric Delta
Base Operating System	Debian GNU/Linux 12 (bookworm v12.15)	Alpine Linux 3.24.3 (musl libc)	Minimal attack surface, zero glibc/systemd exposure
Container Image Size	1,130 MB (1.13 GB)	**457 MB**	**59.5% footprint reduction** (-673 MB)

Runtime Process User	**root (UID 0)**	**node (UID 1000)**	Mitigates container breakout to host system
Build Architecture	Single-stage, dev tools & cache included	Multi-stage build, prod artifacts only	Zero build tooling/compilers in runtime container
OS Vulnerabilities (Total)	**479 vulnerabilities**	**0 vulnerabilities**	**100% OS vulnerabilities eliminated**
OS Vulnerabilities (CRITICAL)	**20 CRITICAL**	**0 CRITICAL**	**100% Critical OS CVEs eliminated**
OS Vulnerabilities (HIGH)	**459 HIGH**	**0 HIGH**	**100% High OS CVEs eliminated**
Total Vulnerabilities (OS + App)	524 (503 HIGH, 21 CRITICAL)	**37 (36 HIGH, 1 CRITICAL)**	**92.9% Total CVE Reduction** (-487 CVEs)



2(d) Domain Threat Modeling, Security Controls & Regulatory Compliance

Threat Agent Analysis & Implemented Technical Controls:

Threat Agent	Motivations & Attack Vector	Implemented Mitigations / Technical Controls
Threat Agent 1: External Cybercriminal / Credential Stuffing Botnet	Automated brute-force attacks against student accounts using credential dumps from third-party breaches to hijack scholarship disbursements.	• **bcrypt (cost 12)** with per-user salt prevents rainbow tables. • **TOTP MFA** halts unauthorized logins even if password is compromised. • Nginx rate limiting throttles automated brute-force scripts.
Threat Agent 2: Malicious Insider / Privilege Escalating Student	A registered student intercepting API traffic and modifying HTTP requests to alter eligibility cutoffs or award status via admin endpoints.	• **Strict RBAC Guards** verify JWT claims and database roles. • PUT / DELETE endpoints strictly reject non-admin tokens with 403 Forbidden. • SSL-encrypted database transport.

Regulatory Compliance Mapping:

• Digital Personal Data Protection Act (DPDPA 2023, India):

Section 8(5) mandates that data fiduciaries implement reasonable security safeguards to prevent personal data breaches. Because our platform collects income certificates and category data of students (often minors), Section 9 child data protections apply. Enforcing bcrypt password hashing, non-root container isolation, and TLS encryption over pooler port 6543 satisfies DPDPA reasonable safeguard requirements.

• General Data Protection Regulation (GDPR, Article 32 — Security of Processing):

Article 32(1)(a) requires pseudonymisation and encryption of personal data; 32(1)(b) requires the ability to ensure confidentiality, integrity, availability, and system resilience; 32(1)(c) mandates rapid restoration of availability in physical/technical incidents. Our multi-instance Nginx failover configuration directly guarantees resilience and zero-downtime recovery under node failure.

Conclusion & Submission Verification

All requirements specified in the evaluation rubric have been fully implemented, validated against the live database, and published:

- **GitHub Repository:** https://github.com/kanishka-netizen/scholarship-platform/tree/assignment-3
- **Load Balancing & Scaling:** 3 NestJS instances behind Nginx with +186.8% throughput improvement.
- **High Availability:** Zero-downtime failover tuning eliminating 100% of node failure errors (from 15 to 0).
- **Cryptographic Storage:** Salted bcrypt password hashing verified in Supabase PostgreSQL.
- **Identity & Access Control:** RFC 6238 TOTP 2FA and NestJS RBAC Guards (401/403/200).
- **Vulnerability Elimination:** Trivy verified 100% elimination of operating system vulnerabilities (from 479 to 0).

Three-Minute Evaluator Video Demonstration Script

Structured 4-phase walkthrough script designed for the required 3-minute video submission and live evaluator assessment:

Time Window	Phase & Objective	Spoken Script & Live Action on Screen
0:00 - 0:45 (45 sec)	**Architecture & Load Balancing:** Multi-instance scaling & round-robin verification	*'Hello everyone. Today I am demonstrating the scalability and security architecture for our Scholarship Discovery Platform. During national scholarship deadlines, thousands of students query eligibility criteria simultaneously. Here, we deployed three NestJS instances behind an Nginx load balancer connecting to our live Supabase PostgreSQL database. As you can see from our curl tests, each request alternates across backend-1, backend-2, and backend-3 via the X-Backend-Instance header. In our 1,000-request k6 benchmark, scaling from 1 to 3 instances improved throughput by 186.8% from 26.5 to 76.2 requests/sec, and slashed average latency from 745ms down to 259ms.'*
0:45 - 1:30 (45 sec)	**Fault Tolerance:** Mid-flight failure & Zero-Downtime Failover	*'Next is fault tolerance. During an active 600-request load test, we abruptly killed backend-2 using docker stop. Under standard Nginx configuration, 15 requests crashed with HTTP 502 Bad Gateway. However, by tuning Nginx with proxy_next_upstream for timeouts and 5xx errors alongside passive health monitoring, Nginx instantly and transparently rerouted in-flight connections to surviving instances. The result: 600 out of 600 requests succeeded with exactly 0% packet loss and zero downtime.'*
1:30 - 2:15 (45 sec)	**Authentication & RBAC:** bcrypt, TOTP MFA, and role authorization	*'Turning to security: Student income and affirmative action certificates are highly sensitive under India's DPDPA 2023. We replaced insecure plaintext storage with salted bcrypt hashing using a cost factor of 12 in Supabase. We also implemented RFC 6238 TOTP two-factor authentication compatible with Google Authenticator, rejecting invalid codes. Finally, our NestJS RBAC Guard protects administrative endpoints: unauthenticated calls return 401, student tokens are rejected with 403 Forbidden, and only Admin tokens are granted 200 OK access.'*
2:15 - 3:00 (45 sec)	**Container Hardening:** Alpine base, non-root user & Trivy scans	*'Lastly, we hardened our Docker container. We transitioned from an unhardened Debian Node 22 image to a multi-stage Alpine build running as low-privilege user node. This slashed image footprint from 1.13 GB down to 457 MB. More importantly, our Trivy vulnerability scan reveals that while the unhardened baseline suffered 479 OS vulnerabilities including 20 CRITICAL and 459 HIGH CVEs, the hardened Alpine image completely eliminated 100% of OS vulnerabilities, recording exactly ZERO critical and ZERO high OS findings. All test logs, diagrams, and code are live on GitHub branch assignment-3. Thank you.'*